



WEB APPLICATION DEVELOPMENT-I

UNIT-1

SHORT QUESTIONS:

✓ **1/ What is the importance of frontend development in web design?**

The importance of Web/Frontend development lies in its direct impact on how users interact with a website or web application.. It ensures the site is user-friendly, responsive, and visually appealing, providing a smooth user experience across different devices and browsers.

✓ **2/ List any four basic HTML elements.**

<html> – Defines the root of an HTML document.

<head> – Contains metadata, title, and links to styles or scripts.

<body> – Contains the visible content of the webpage.

<p> – Defines a paragraph of text.

✓ **3/ What is the purpose of the <head> and < body> tags in HTML?**

The <head> tag contains metadata, title, links to CSS, and scripts that define the webpage's structure and behavior. The <body> tag holds the visible content such as text, images, and links displayed to users. Together, they separate the document's setup from its displayed content. The <head> runs behind the scenes, while <body> is what users interact with.

✓ **4/ Which tag is used to link CSS with HTML?**

The <link> tag is used to connect an external CSS file with an HTML document. It is placed inside the <head> section of the HTML page. The rel="stylesheet" attribute defines the relationship between the HTML and CSS file. For example: <link rel="stylesheet" href="style.css">.

✓ **5/ Where is the <style> tag usually placed?**

The <style> tag is usually placed inside the <head> section of an HTML document. It allows you to write internal CSS directly within the HTML file. Styles written here apply to the elements on that same page. It helps in quick styling without using an external CSS file.

✓ **6/ What is inheritance in CSS?**

Inheritance in CSS means that some CSS properties applied to a parent element are automatically **passed down (inherited)** by its child elements.

✓ **7/ What is the difference between padding and margin?**

PADDING	MARGINS
Padding is the space inside an element, between its content and its border.	The margin properties are used to create space around elements, outside of any defined borders.

✓ **8/ Differentiate between class selector and ID selector in CSS.**

A class selector (e.g., .example) styles multiple elements with the same class. An ID selector (e.g., #example) is unique and applied to only one element per page. Classes promote reusability, while IDs identify specific elements. IDs have higher specificity than classes in CSS priority.

LONG QUESTIONS:

✓ 1 Explain different types of HTML elements such as headings, paragraphs, images, buttons with examples.

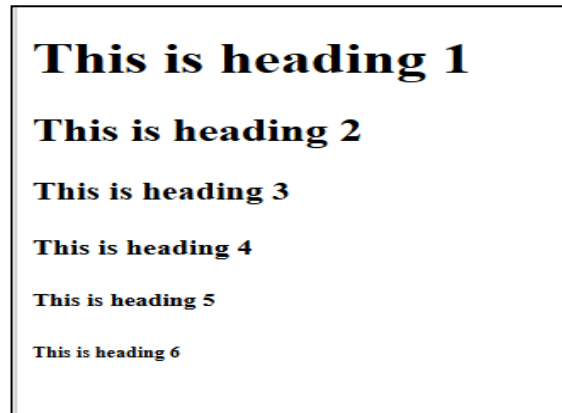
HTML Headings

HTML headings are defined with the <h1> to <h6> tags.

<h1> defines the most important heading. <h6> defines the least important heading:

Example

```
<!DOCTYPE html>
<html>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```



HTML Paragraphs:

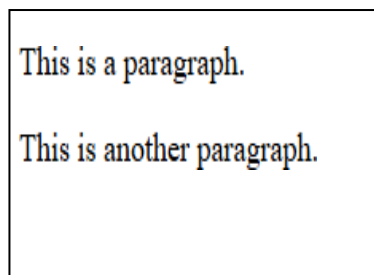
HTML paragraphs are defined with the <p> tag

Example:

```
<!DOCTYPE html>
<html>
<body>

<p>This is a paragraph.</p>
<p>This is another paragraph.</p>

</body>
</html>
```



HTML Buttons

The <button> tag in HTML is used to create clickable buttons on a webpage.

These buttons can trigger various actions, such as submitting a form, or simply interacting with users.

Syntax

```
<button>Click Me!</button>
```

The opening <button> tag marks the start of the button element.

The closing </button> tag indicates the end of the button element.

The text Click Me! will be displayed on the button.

Example

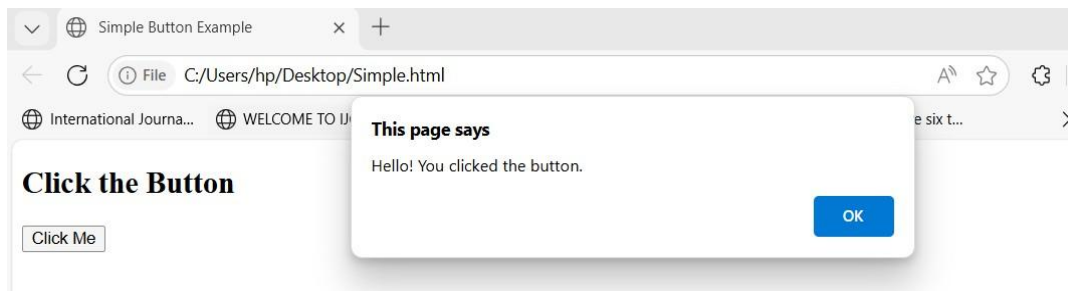
```
<!DOCTYPE html>
<html>
<head>
<title>Simple Button Example</title>
</head>
<body>
```

<h2>Click the Button</h2>

<button onclick="alert('Hello! You clicked the button.')">Click Me</button>

</body>

</html>



HTML IMAGES

Images can improve the design and the appearance of a web page.

In HTML, the tag is used to display an image on a web page.

The tag has two required attributes:

src - Specifies the path to the image

alt - Specifies an alternate text for the image

Example

src → Path or URL of the image file

alt → Text shown if the image cannot be displayed (for accessibility)

width & height → Control the size of the image

Example:

!DOCTYPE html>

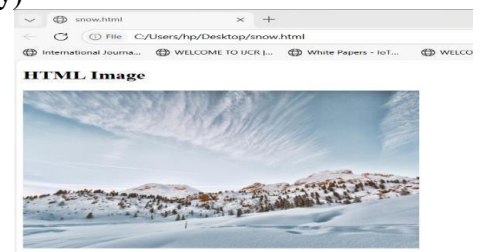
<html>

<body>

<h2>HTML Image</h2>

</body>

</html>



2/ Discuss the difference between block-level and inline elements with examples.

Block Elements:

Block-level elements always begin on a new line, effectively creating a "block" of content.

These are Take up the full available width (i.e stretch from left to right).

It Can contain block and inline elements.

These are used to structure the layout of a webpage.

Examples: <div>, <p>, <h1> to <h6>, , , , <header>, <footer>, <section>, <article>.

Inline Elements:

Do not start on a new line: Inline elements flow within the existing line of content.

Take up only necessary width: They only occupy as much horizontal space as their content requires.

Cannot have explicit width and height: Setting width and height properties on inline elements generally has no effect.

Only horizontal margins and padding: Vertical (top and bottom) margins and padding are typically ignored, though horizontal margins and padding are applied.

Examples: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`, `<input>`, `<label>`, `<code>`.

Differences: Feature	Block Elements	Inline Elements
Width	Full width of parent	Only as wide as content
Line Break	Starts on a new line	Does not start on new line
Containment	Can hold block + inline	Can hold only text/inline
Usage	Page structure/layout	Style small parts of text

✓ 3. Write short notes on the following: a) Padding and Margin b) CSS Units c) HTML Containers.

a) Padding and Margin

Padding:

- Padding is the **space between an element's content and its border**.
- It increases the inner spacing within an element.
- Example:

```
p {  
padding: 20px;  
}
```

→ Adds 20px space inside the paragraph border.

Margin:

- Margin is the **space outside an element's border**, separating it from other elements.
- Example:

```
p {  
margin: 20px;  
}
```

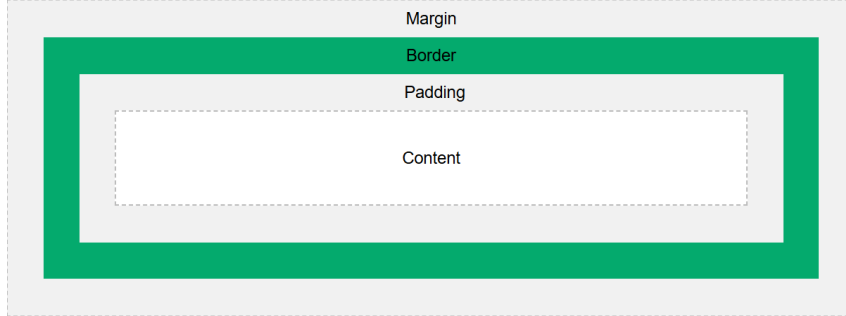
→ Adds 20px space outside the paragraph.

b) CSS Units

CSS units define **measurements** such as size, spacing, and positioning. They are classified into two types:

Absolute units: Fixed sizes (do not change with screen size) Examples: px (pixels), pt (points), cm, mm, in

Relative units: Adjust based on parent or viewport



Examples: em (relative to font size), rem (root font size), % (percentage), vw (viewport width), vh (viewport height)

c) HTML Containers

HTML containers are elements used to **group and organize** other HTML elements. They help structure the webpage and apply styling or layout.

- A **container** is an HTML element that is used to group or hold other HTML elements together on a web page
- It helps organize the layout and apply common styles to multiple elements at once.

Types of Containers

- **<div>** → A generic block container (most common).
- **<section>** → Groups related content into a section.
- **<article>** → For independent content (blog post, news article).
- **<header>** → Top part of a page or section (logo, navigation).
- **<footer>** → Bottom part of a page or section (contact, copyright).
- **** → An inline container for small parts of text.

Example:

```
<div class="container">
<h2>Welcome</h2>
<p>This is inside a container.</p>
</div>
```

4 ✓ Write and explain the CSS box model with a neat diagram.

Every HTML element is like a box.

The Box Model describes how size and spacing of an element is calculated.

It has 4 parts (from inside → outside):

1. Content → Text, images inside the box
2. Padding → Space between content and border
3. Border → Line around the element
4. Margin → Space outside the border (separates from other elements)

Explanation of the different parts:

Content - The content of the box, where text and images appear

Padding - Clears an area around the content. The padding is transparent

Border - A border that goes around the padding and content

Margin - Clears an area outside the border. The margin is transparent

5. Explain the importance of debugging and tweaking code in web development.

The core fundamentals of programming involve understanding debugging, and the ability to tweak code for desired functionality. These are essential skills for any programmer, regardless of experience level. Debugging is the process of finding and fixing errors in the code, and tweaking code involves making small adjustments to existing code to improve its performance, readability, or functionality.

1. Debugging:

- Debugging is the process of identifying, isolating, and correcting errors (also known as “bugs”) in a program’s code.
- Debugging can involve various techniques, including:
 - **Code Reviews :** Having another person review your code can help catch errors you might have missed.
 - **Print Statements:** Adding print statements strategically throughout the code can help track the flow of execution and identify where things go wrong.
 - **Debuggers:** Debuggers are tools that allow you to step through code line by line, inspect variables, and identify the exact point where an error occurs.
- Effective debugging involves:
 - Reproducing the bug consistently.
 - Understanding the error message.
 - Isolating the source of the error.
 - Correcting the error.
 - Testing the fix to ensure it works as expected.

2. Tweaking Code:

- Tweaking code involves making small changes to existing code to improve its behavior or functionality.
- This can include:
 - **Optimizing Performance:** Making code run faster or use less memory.
 - **Improving Readability:** Making code easier to understand and maintain.
 - **Adding New Features:** Implementing new functionality or behavior.
- Tweaking code often involves:
 - **Understanding the Existing Code:** You need to understand how the existing code works before you can effectively tweak it.
 - **Testing Changes:** Always test any changes you make to ensure they work as expected and don’t introduce new problems.
 - **Iterative Approach:** Tweaking code is often an iterative process, where you make small changes, test, and then refine them further.

✓ 6. Define inheritance and specificity in CSS and explain how they affect styling.

CSS Inheritance

HTML itself does not have a concept of “inheritance” in the same way that object-oriented programming languages like Java or Python do, where one class can inherit methods and properties from another.

However, the term “inheritance” in the context of HTML primarily refers to CSS inheritance, which describes how styling properties are passed down from parent elements to their child elements in the HTML document tree.

In CSS, **inherit** is a special keyword used to tell an element to take the same property value as its parent element.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.parent {
color: red; /* Parent text color is red */
}
.child {
color: inherit; /* Child takes the color from parent */
}
</style>
</head>
<body>
<div class="parent">
I am red.
<p class="child">I will also be red (inherited).</p>
</div>
</body>
</html>
```

Output:

I am red.

I will also be red (inherited).

Here, .parent has color: red and .child has color: inherit, so it gets the same red color from .parent

Specificity

If there are two or more CSS rules that point to the same element, the selector with the highest specificity will “win”, and its style declaration will be applied to that HTML element.

Think of specificity as a hierarchy that determines which style declaration is ultimately applied to an element.

Specificity Hierarchy

Every CSS selector has a position in the specificity hierarchy.

Priority	Example	Description
Inline style	<code><h1 style="color: pink;"></code>	In line styles Hierarchy : ID selectors (#id) Class selectors (.class), attributes, pseudo-classes Element selectors (p, div) and pseudo-elements Universal selector (*)
Idselectors	<code>#navbar</code>	
Classes and pseudo-classes	<code>.test, :hover</code>	
Attributes	<code>[type="text"]</code>	

Elements and pseudo-elements	h1,::before, ::after	Lowest priority,applies to HTML elements and pseudo-elements
-------------------------------------	-----------------------------	--

Example:

```

<!DOCTYPE html>
<html>
<head>
<style>
/* Lowest Priority — Element Selector */
h1 {
color: red;
}
/* Attribute Selector — slightly higher */
h1[title="heading"] {
color: orange;
}
/* Class Selector */
.highlight {
color: green;
}
/* ID Selector */
#mainHeading {
color: blue;
}
</style>
</head>
<body>
<!--Inline Style has Highest Priority -->
<h1 id="mainHeading" class="highlight" title="heading" style="color: pink;">
CSS Priority Demo
</h1> <p>Check which color is applied to the heading!</p>
</body>
</html>

```

OUTPUT:



- ✓ **Describe the different types of lists in HTML (ordered, unordered, and description lists). Explain with an example.**

An **HTML List** allows you to organize data on web pages into an ordered or unordered format to make the information easier to read and visually appealing. HTML Lists are very helpful for creating structured, accessible content in web development.

Types of HTML Lists

There are three main types of lists in HTML:

- 1. Unordered Lists ():** These lists are used for items that do not need to be in any specific order. The list items are typically marked with bullets.
- 2. Ordered Lists ():** These lists are used when the order of the items is important. Each item in an ordered list is typically marked with numbers or letters.

3. Description Lists (<dl>): These lists are used to contain terms and their corresponding descriptions.

HTML List Tags Tag	Description
	Defines an unordered list.
	Defines an ordered list.
	Defines a list item.

1. Using HTML Unordered List or Bulleted List

Unordered lists are ideal for scenarios where the sequence of the items is not important.

The unordered list items are marked with bullets, also known as bulleted lists. An unordered list starts with the tag, and each list item begins with the tag.

Syntax:

```
<ol>
<li>Item1</li>
<li>Item2</li>
<li>Item3</li>
</ol>
```

2. Using HTML Ordered List

Ordered lists are used when the items need to follow a specific sequence.

In an ordered list, all list items are marked with numbers by default. An ordered list starts with the tag, and each list item begins with the tag.

3. Using HTML Description List

A description list is a list of terms, with a description of each term. Description lists are less common but very useful for definitions, glossaries, or any other key-value pairs of items.

The <dl> tag defines the description list, the <dt> tag defines the term name, and the <dd> tag describes each term.

Syntax:

```
<dl> <dt>Item1</dt> <dd>DescriptionofItem1</dd> <dt>Item2</dt> <dd>DescriptionofItem2</dd> </dl>
```

Here, <dt> (description term) is used for the term being defined, and <dd> (description details) is used for the description.

Example:

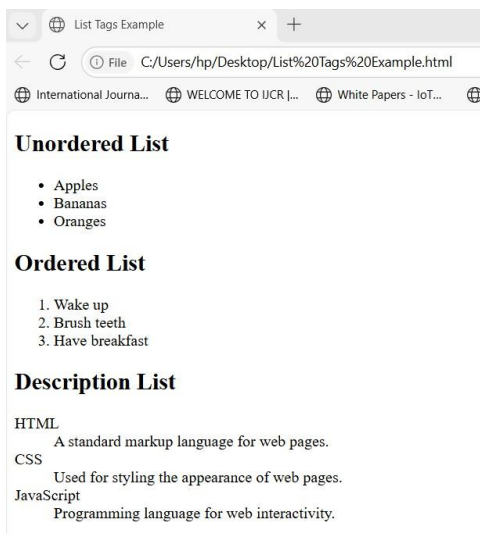
```
<!DOCTYPE html>
<html>
<head>
<title>List Tags Example</title>
</head>
<body>
<h2>Unordered List</h2>
<ul>
<li>Apples</li>
<li>Bananas</li>
<li>Oranges</li>
</ul>
<h2>Ordered List</h2>
<ol>
<li>Wake up</li>
<li>Brush teeth</li>
```

```

<li>Have breakfast</li>
</ol>
<h2>Description List</h2>
<dl>
<dt>HTML</dt>
<dd>A standard markup language for web pages.</dd>
<dt>CSS</dt>
<dd>Used for styling the appearance of web pages.</dd>
<dt>JavaScript</dt>
<dd>Programming language for web interactivity.</dd>
</dl>
</body>
</html>

```

Output:



8. Differentiate between absolute and relative CSS units.

CSS UNITS:

CSS units define the size of elements, with absolute units (like px, cm) having fixed values and relative units (like em, rem, %, vh) depending on factors like the viewport or parent elements.

There are two types of units: Absolute and Relative.

Absolute Units in CSS

In CSS, **absolute units** are fixed and do **not scale** based on screen size or user preferences. These units have a defined physical size and are generally used when you need **precise control** over layout or print styles. However, they're **not responsive**, so they are best used sparingly in modern web design.

Common Absolute Units in CSS Unit	Full Form	Description
px	Pixels	The most common unit. Represents a dot on the screen. Not truly physical but widely supported.
pt	Points	Common in print. 1 pt = 1/72 inch. Used in typography.
pc	Picas	1 pc = 12 pt = 1/6 inch. Mostly used in print layout.
in	Inches	1 inch = 2.54 cm = 96 px. Mostly used for print styles.
cm	Centimeters	1 cm = 10 mm = ~37.8 px.

mm	Millimeters	10 mm = 1 cm.
----	-------------	---------------

Relative Units in CSS

In CSS (Cascading Style Sheets), **relative units** are units of measurement that scale based on other elements or properties, making them flexible and responsive. They are especially useful in modern **responsive design** where content must adapt to different screen sizes and resolutions.

Unit	Meaning	Based On	Example
em	Relative to the font-size of the parent element	Parent element's font-size	padding: 2em;
Unit	Meaning	Based On	Example
rem	Relative to the root element's font-size (usually <html>)	Root (<html>) font-size (default is 16px)	font-size: 1.5rem;
%	Relative to the parent element (depends on property: width, height, etc.)	Parent element's dimensions	width: 80%;
vh	1% of the viewport height	Browser window's height	height: 100vh;
vw	1% of the viewport width	Browser window's width	width: 50vw;
vmin	1% of the smaller dimension of viewport (height or width)	Minimum of vh and vw	font-size: 5vmin;
vmax	1% of the larger dimension of viewport (height or width)	Maximum of vh and vw	font-size: 5vmax;
ch	Relative to the width of the 0 character of the element's font	Width of character 0	width: 30ch;
ex	Relative to the x-height of the element's font (height of lowercase x)	x-height of font	line-height: 2ex;

UNIT-2

SHORT QUESTIONS:

1. What is the purpose of the Bootstrap Grid System?

✓ **BOOTSTRAP GRID SYSTEM** :The Bootstrap grid system is a powerful tool for building responsive web layouts. It's based on a 12-column structure, allowing you to easily arrange and size your content.

2. What are Flex container properties?

Flex container properties are the CSS properties applied to a flex container (an element with display: flex or display: inline-flex) to control the layout of its child items. The main flex container properties are:

flex-direction, flex-wrap, justify-content, align-items, align-content, and gap.

These properties define the direction, alignment, spacing, and wrapping of flex items inside the container.

3. What is the difference between .container and .container-fluid in Bootstrap?

Creates a **fixed-width** layout that adjusts at different breakpoints (responsive but not full width).
.container-fluid

Creates a **full-width** layout that always spans the **entire width** of the screen.

✓ **4. What are Bootstrap sizing utilities used for?**

Bootstrap sizing utilities are used to **control the width and height** of elements quickly and easily using predefined utility classes — without writing custom CSS.

✓ **5. What is the use of the Bootstrap colour system?**

Bootstrap's color system provides predefined contextual color classes (like `.text-primary`, `.bg-success`). It ensures design consistency and quick styling using standardized colors. Developers can apply colors to text, backgrounds, and borders easily. It supports customization through Bootstrap's SCSS variables.

✓ **6. Define align-items and align-self in Flexbox.**

`Align-items` aligns all flex items along the cross-axis (vertically by default). It applies a common alignment for all items within a flex container. `Align-self` overrides `align-items` for a specific item only. Both help control vertical alignment and spacing in flexible layouts.

✓ **7. What is the difference between m and p classes?**

In Bootstrap, **m** stands for margin and **p** stands for padding. Margin creates space outside an element's border, while padding creates space inside it. For example, `m-3` adds margin and `p-3` adds padding. Both use spacing utilities to control layout spacing easily.

✓ **8. What is the difference between .bg-primary and .text-primary?**

`.bg-primary` sets the background color to the theme's primary color. `.text-primary` changes the text color to the primary color. They both use Bootstrap's predefined color palette. These classes help maintain consistent design and color usage across a website.

LONG QUESTIONS:

1. Explain Dynamic Elements (CAROUSEL) along with its parts. Explain with one example?

DYNAMIC ELEMENTS (CAROUSEL)

The Carousel plugin is a component for cycling through elements, like a carousel (slideshow). Bootstrap has a Carousel component that allows you to create sliding image or content sliders. It supports images, text, buttons, captions, controls, and indicators.

Bootstrap Carousel Parts

1. carousel slide Class that initializes the carousel and applies the slide animation.

2. data-bs-ride="carousel"

Enables automatic sliding of the carousel.

If omitted, the user has to click arrows/dots to move slides.

3. carousel-inner

Wrapper that holds all the slides.

4. carousel-item

Each individual slide (image, card, or content).

You must have at least one inside `carousel-inner`.

5. active

Applied to the **first** (or currently visible) slide.

Without it, no slide would show by default.

6. carousel-indicators

The small dots at the bottom that allow navigation between slides.

Each button corresponds to a slide via `data-bs-slide-to`.

7. carousel-control-prev & carousel-control-next

Left and right arrow buttons for manual navigation.

They use data-bs-slide="prev" or "next" to switch slides.

Example:

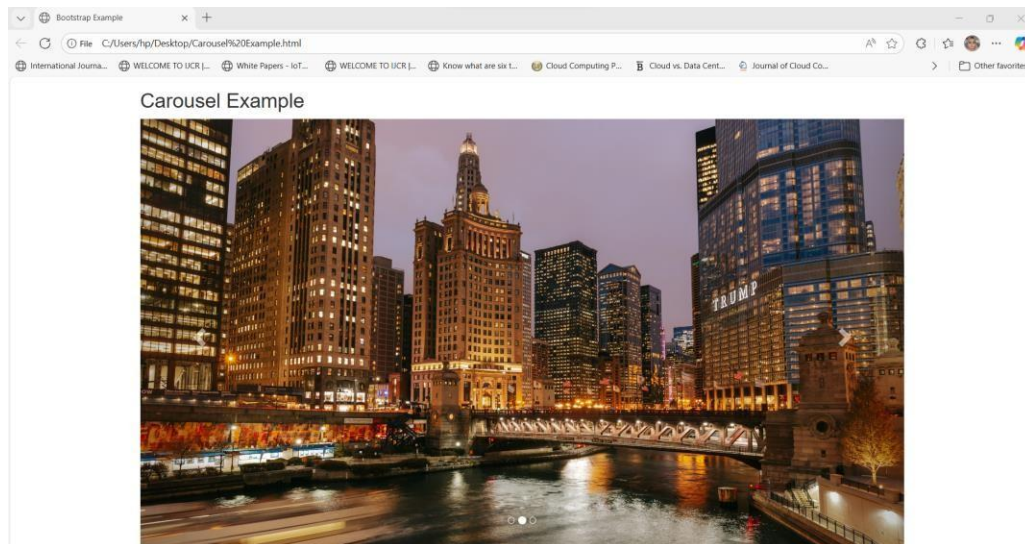
```
<!DOCTYPE html>
<head>
<title>Bootstrap 5 Carousel Example</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<!-- Bootstrap 5 CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container mt-4">
<h2>Bootstrap 5 Carousel Example</h2>
<div id="myCarousel" class="carousel slide" data-bs-ride="carousel">
<!-- Indicators -->
<div class="carousel-indicators"> <button type="button" data-bs-target="#myCarousel" data-bs-slide-to="0" class="active"></button>
<button type="button" data-bs-target="#myCarousel" data-bs-slide-to="1"></button>
<button type="button" data-bs-target="#myCarousel" data-bs-slide-to="2"></button>
</div>
<!-- Wrapper for slides -->
<div class="carousel-inner">
<div class="carousel-item active">

</div>
<div class="carousel-item">

</div>
<div class="carousel-item">

</div>
</div>
<!-- Left and right controls -->
<button class="carousel-control-prev" type="button" data-bs-target="#myCarousel" data-bs-slide="prev">
<span class="carousel-control-prev-icon"></span>
<span class="visually-hidden">Previous</span>
</button>
<button class="carousel-control-next" type="button" data-bs-target="#myCarousel" data-bs-slide="next">
<span class="carousel-control-next-icon"></span>
<span class="visually-hidden">Next</span>
</button>
</div> </div>
<!-- Bootstrap 5 JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Output:



EXPLANATION OF PROGRAM:

1. Start Carousel Box

```
<div id="myCarousel" class="carousel slide" data-bs-ride="carousel">
```

- Makes a box for the slideshow.
- id="myCarousel" → name of this carousel.
- carousel slide → tells Bootstrap this is a sliding carousel.
- data-bs-ride="carousel" → makes it move automatically.

2. Dots (Indicators)

```
<div class="carousel-indicators"> <button data-bs-slide-to="0" class="active"></button>
```

```
<button data-bs-slide-to="1"></button>
```

```
<button data-bs-slide-to="2"></button>
```

```
</div>
```

- The **3 dots** at the bottom.
- Click them to go to slide 1, 2, or 3.
- active → means the first dot is selected.

3. Slides (Images)

```
<div class="carousel-inner">
```

```
<div class="carousel-item active">
```

```

```

```
</div>
```

```
<div class="carousel-item">
```

```

```

```
</div>
```

```
<div class="carousel-item">
```

```

```

```
</div>
```

```
</div>
```

- This is the **main part** (the pictures).
- carousel-inner → holds all the slides.
- carousel-item → one slide.
- active → first slide shows first.
- img → your picture (100% width).

4. Arrows (Controls)

```
<button class="carousel-control-prev" data-bs-slide="prev">
<span class="carousel-control-prev-icon"></span>
</button>
<button class="carousel-control-next" data-bs-slide="next"> <span class="carousel-control-next-
icon"></span>
</button>
```

- Left arrow → goes to previous picture.
- Right arrow → goes to next picture.
- Bootstrap gives you arrow icons automatically.

5. Bootstrap JS

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

- Makes the carousel work (sliding, arrows, dots).
- No jQuery needed in Bootstrap 5.

2. Explain foundational principles of responsive design?

Flexbox uses principles like flexible sizing and automatic wrapping with properties such as flex-wrap: wrap to make content adapt to different screen sizes, ensuring a consistent user experience across devices. Combined with media queries, you can apply different flexbox behaviors—like changing flex-direction from row to column—at specific breakpoints, allowing layouts to reshape and resize fluidly for optimal viewing on mobile, tablet, and desktop views.

Here's a breakdown of the responsive design principles flexbox utilizes:

Flexible Item Sizing

- **flex-grow, flex-shrink, flex-basis:** These properties control how flex items can grow or shrink to fill available space, or what their initial size should be.
- **Fluid Layouts:** Instead of fixed pixel values, Flexbox allows items to use relative units, making the entire layout fluid and adaptable to various screen dimensions.

Automatic Wrapping

flex-wrap: wrap: This property is a key principle for responsiveness. When there isn't enough horizontal space for items in a row, they will automatically move to the next line, effectively stacking them vertically.

Media Queries for Breakpoints

- **Device-Specific Styles:** Media queries allow you to apply specific CSS rules when certain conditions are met, such as the browser window's width falling within a particular range.
- **Adapting Layouts:** You can use media queries to change flex-direction from a horizontal row (desktop view) to a vertical column (mobile view), making elements stack on smaller screens.

Content Alignment

- **justify-content & align-items:** Flexbox offers powerful properties for controlling the alignment of items along the main and cross axes.
- **Consistent Centering:** These properties ensure content remains centered or evenly distributed, regardless of screen size, providing a polished and consistent presentation.

By combining these core Flexbox properties with the foundational principles of responsive design, such as fluid grids and media queries, developers can create websites that dynamically adjust and provide a seamless, user-friendly experience on any device.

3. Describe the container, row, and column structure in Bootstrap with examples.

The Bootstrap grid system is a powerful tool for building responsive web layouts. It's based on a 12-column structure, allowing you to easily arrange and size your content. By using its predefined classes, your website will effortlessly adapt to various screen sizes, from desktops to mobile phones.

Components of Grid System:

• 12-Column Structure:

The core of the Bootstrap grid is a 12-column system. Content is placed within columns, and the width of these columns is defined by how many of the 12 available columns they span. The sum of column spans within a single row should ideally add up to 12 or less.

• Containers:

Content is typically placed within a `.container` (fixed-width) or `.container-fluid` (full-width) to ensure proper alignment and padding. **Example:** `<div class="container">`

...

`</div>`

• Rows:

Rows (`.row`) are used to create horizontal groups of columns. Content should be placed within columns, and only columns may be direct children of rows.

Example: `<div class="row">`

...

`</div>`

- Columns: o xs (extra-small, no infix in Bootstrap 5)

o sm (small)

o md (medium)

o lg (large)

o xl (extra-large)

o xxl (extra-extra-large, in Bootstrap 5)

• Columns:

Columns (`.col-*`) define the width and responsiveness of content blocks. Bootstrap provides predefined classes for various screen sizes (breakpoints):

Example: `<div class="container">`
`<div class="row">`
`<div class="col-md-6">Left Column</div>`
`<div class="col-md-6">Right Column</div>`
`</div>`
`</div>`

Here, col-md-6 means the column will take 6 out of 12 grid spaces when the screen size is medium ($\geq 768\text{px}$) or larger. The grid in Bootstrap is based on 12 columns, so 6 means the column takes half the width.

4. Explain alignment properties in Flexbox – align-items, align-self, and justify-content.

Flexbox is a layout model in bootstrap that is used to design responsive mobile-friendly layouts of a webpage. Unlike CSS grids, flexboxes are used to design one-dimensional layouts of a page. The flexboxes are used to align the elements in a container even if we don't know the sizes of the elements. This is because a flex container is flexible and therefore, it expands or shrinks the elements to fill up the space or avoid the overflow inside the container. Below are the few alignment properties, they are:

Justify Content (Main Axis)

Controls horizontal alignment (if row) or vertical alignment (if column). Class	Effect
justify-content-start	Align left (default)
justify-content-center	Center
justify-content-end	Align right
justify-content-between	Items spread with space between
justify-content-around	Equal space around items
justify-content-evenly	Equal space between items

Example:

```
<div class="d-flex justify-content-between bg-light p-2">
<div class="p-2 bg-primary text-white">Left</div>
<div class="p-2 bg-success text-white">Right</div>
</div>
```

2. Align Items (Cross Axis)

Controls alignment perpendicular to direction (vertical if row, horizontal if column). Class	Effect
align-items-start	Align to top/left
align-items-center	Center
align-items-end	Align to bottom/right
align-items-stretch	Stretch to container height (default)
Class	Effect
align-items-baseline	Align based on text baseline

Example:

```
<div class="d-flex align-items-center bg-light" style="height:100px;">
<div class="p-2 bg-warning">Short</div>
<div class="p-4 bg-info">Tall</div>
</div>
```

3. Align Self (Per Item Control)

Instead of aligning all, you can align a	Effect
--	---------------

single item. Class	
align-self-start	Align this item to start
align-self-center	Center this item
align-self-end	Align this item to end
align-self-stretch	Stretch this item
align-self-baseline	Align to baseline

Example:

```
<div class="d-flex bg-light" style="height:100px;">
<div class="p-2 bg-primary text-white align-self-start">Top</div>
<div class="p-2 bg-success text-white align-self-center">Middle</div>
<div class="p-2 bg-danger text-white align-self-end">Bottom</div>
</div>
```

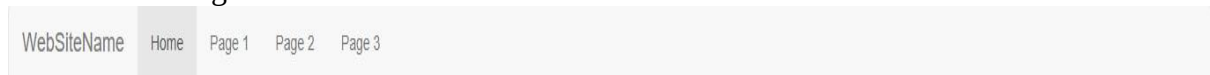
5. Write notes on Bootstrap UI Components like Navbar and Buttons with examples.

A navigation bar is a navigation header that is placed at the top of the page:

Navigation bars are an important component of web design. It helps users to easily navigate between different sections of a website.

Navigation bars are typically built with HTML list elements (and).

A standard navigation bar is created with **<nav class="navbar navbar-default">**.

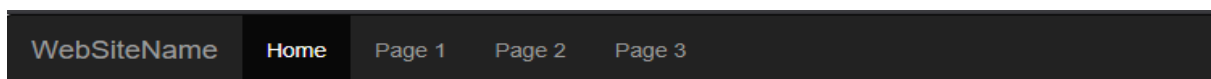


Basic Navbar Example

A navigation bar is a navigation header that is placed at the top of the page.

Bootstrap provides an alternative, black navbar: just change the .navbar-default class into .navbar-inverse:

<nav class="navbar navbar-inverse">



Basic Navbar Example

A navigation bar is a navigation header that is placed at the top of the page.

Bootstrap Buttons

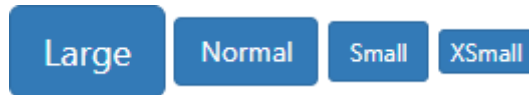
Bootstrap provides different styles of buttons:



To achieve the button styles Bootstrap has the following classes:

- .btn
- .btn-default
- .btn-primary
- .btn-success
- .btn-info
- .btn-warning
- .btn-danger

- .btn-link **Button Sizes**
Bootstrap provides four button sizes:



The classes that define the different sizes are:

.btn-lg

btn-primary

.btn-sm

.btn-xs

Syntax

For Default button

```
<button type="button" class="btn btn-default">Default</button>
```

For Small size button

```
<button type="button" class="btn btn-primary btn-sm">Small</button>
```

6. Explain media integration in Bootstrap (images, videos, responsive embeds). Media Integration.

Media Integration

- Media integration means adding multimedia elements like images, audio, and video into a web page.
- It Enhances user experience.
- It Makes content more interactive and engaging.
- HTML provides built-in tags: → Images

<audio> → Audio files

<video> → Video files

<iframe> → Embedding external media (like YouTube). Explanation of Media Tags

1. Image Integration ()
 - Displays an image in a webpage.
 - Attributes: src (source), alt (alternate text), width, height.
2. Audio Integration (<audio>)
 - Embeds audio files like .mp3, .ogg.
 - Attributes: controls (play, pause, volume), autoplay, loop.
3. Video Integration (<video>)
 - Embeds video files like .mp4, .webm.
 - Attributes: controls, autoplay, muted, loop, poster.
4. External Media (<iframe>)
 - Used to embed content from another website (e.g., YouTube, Google Maps).
5. displays a picture from an online source.
6. <audio> lets users **play/pause** an audio file.

7. <video> embeds a video with controls.
8. <iframe> loads YouTube video inside the page.

7. Write short notes on: a) Bootstrap spacing utilities b) Flexbox alignment properties c) Bootstrap colour system.

a) Spacing Utilities

Bootstrap spacing utilities allow you to control margin and padding.

Syntax

{property}{sides}-{size}

Property: m = margin, p = padding

Sides: t = top, b = bottom, l = left, r = right, x = left+right, y = top+bottom, blank = all sides **Size:** 0 to 5, auto

Examples

Class	Effect
mt-3	Margintop 1rem
mb-0	Marginbottom 0
px-4	Paddingleft &right 1.5rem
py-2	Paddingtop & bottom 0.5rem
m-auto	Automargin(centerblock)

b) Flexbox Alignment Properties

Purpose: Flexbox makes it easy to align and distribute space among items inside a container.

Common alignment properties:

Property	Description
justify-content	Aligns items horizontally (main axis)
align-items	Aligns items vertically (crossaxis)
align-content	Controls multiple rows' alignment(when wrapping)
align-self	Aligns a single flex item individually

c) Color Systems

Bootstrap provides predefined color utility classes to quickly style elements without writing custom CSS. These include background, text, and border colors.

1. Background Colors (bg-*) Adds color to an element's background.

Example: bg-primary, bg-success, bg-danger, bg-warning, bg-info, bg-secondary, bg-dark, bg-light.

2. Text Colors (text-*) Changes the color of text.

. Example: text-primary, text-success, text-danger, text-warning, text-info, text-secondary, text-dark, text-muted, text-white.

3. Border Colors (border-*) Adds color to borders.

Example: border-primary, border-success, border-danger, border-warning, border-info, border-secondary, border-dark, border-light.

Default color : primary → Blue, secondary → Gray, success → Green, danger → Red, warning → Yellow, info → Cyan, light → Light Gray, dark → Black

8. Explain Bootstrap display and sizing, utilities with examples.

Display Utilities

The display property specifies the display behaviour (the type of rendering box) of an element.

In Bootstrap, the **display property** controls the layout behaviour of elements.

Bootstrap provides utility classes like **d-none (hide)**, **d-block (display as block)**, **d-inline (display as inline)**, etc., for responsive display manipulation. The uses are:

The available classes are:

1) .d-block

Sets the display property of an element to block, making it a block-level element.

2) .d-inline

Sets the display property of an element to inline, allowing it to flow inline with text.

3) .d-inline-block

Sets the display property of an element to inline-block, combining inline and block.

4) .d-flex

Displays content in a box format using the Flexbox layout model.

5) .d-grid

Displays content in a grid format using the CSS grid layout model. **Syntax**

```
<div class="d-inline"> Inline </div> // for inline display
```

```
<div class="d-block"> Block </div> // for block display
```

```
<div class="d-inline-block"> Inline Block </div> // for inline-block display
```

```
<div class="d-flex"> flex box </div> // for flex-box display
```

```
<div class="d-grid"> grid </div> // for grid display
```

■ Responsive Display Classes

Bootstrap allows display changes at different screen sizes:

- **d-none d-sm-block** → hidden on extra small, visible on small+
- **d-none d-md-block** → hidden below medium, visible on medium+
- **d-none d-lg-block** → hidden below large, visible on large+
- **d-block d-md-none** → visible only on small screens

Breakpoints:

- sm ≥ 576px
- md ≥ 768px
- lg ≥ 992px
- xl ≥ 1200px
- xxl ≥ 1400px

Sizing Utilities

Bootstrap sizing utilities control width and height easily.

Common Sizing Classes

Utility	Description
w-25,w-50,w-75,w-100	Width25%,50%,75%,100%
h-25,h-50,h-75,h-100	Height25%,50%,75%,100%
mw-100	Max-width100%

Max-height100%

Minimumwidth/height100%of viewport

Example: Sizing Utilities

<div class="bg-info text-center p-2 h-25 mt-2">Height 25%</div>

LOGICAL OPERATORS IN CSS

When writing **media queries**, sometimes a single condition (like `max-width: 600px`) is not enough. For complex responsive designs, we use **logical operators** to combine or refine multiple conditions.

Types of Logical Operators

- Combines multiple conditions.
- All conditions must be **true** for the styles to apply.
- Example: `@media (min-width: 600px) and (orientation: landscape)`

- Negates a media query.
- Styles apply only if the query condition is false.
- Example: @media not screen → applies to all media types **except screen**.

- Prevents older browsers (that don't understand media queries) from applying styles.
- Commonly used in **mobile-first design**.
- Example: @media only screen and (max-width: 768px)

- Allows multiple queries separated by commas.
- If **any one condition is true**, the styles apply.
- Example: @media (max-width: 500px), (orientation: portrait)

Examples:

```
/* Base style */
body {
font-size: 16px;
background-color: white;
}
/* Use AND operator: screen width ≥ 768px AND orientation is portrait */
@media (min-width: 768px) and (orientation: portrait) {
body {
font-size: 20px;
background-color: lightyellow;
}
}
/* Use OR operator: width ≤ 500px OR landscape orientation */
@media (max-width: 500px), (orientation: landscape) {
body {
font-size: 14px;
background-color: lightgreen;
}
}
```

UNIT-3

SHORT QUESTIONS:

1. Where can media queries be written — inside or outside the CSS file?

Media queries can be written inside a CSS file or within a <style> tag in HTML. They are typically placed inside external CSS files for better organization. They control the layout and styles based on screen size or device type. Using them externally helps maintain responsive and clean code structure.

2. How do you target devices in landscape mode using media queries?

To target devices in landscape mode using CSS media queries, you use the orientation feature.

Syntax

```
@media (orientation: landscape)
{ ...
}.
```

3. List any two logical operators used in media queries

and – Combines multiple conditions.

not – Excludes a specific condition.

4. Define media types in CSS. Give two examples.

In CSS, media types are used in media queries to apply styles based on the type of device or output medium (like screen, print, etc.). They help make web pages responsive and adaptable to different devices.

Examples:

1. @media screen { ... } — applies styles for computer screens, tablets, or phones.
2. @media print { ... } — applies styles when printing the web page.

5. What is a CSS transition?

6. What is CSS animation?

7. What is the difference between transition and animation?

LONG QUESTIONS:

1. Discuss logical operators in CSS (and, not, only, or) with examples.

LOGICAL OPERATORS IN CSS

Logical operators help combine or restrict multiple **media features** (like width, height, device type) to apply styles only under specific conditions.

When writing **media queries**, sometimes a single condition (like max-width: 600px) is not enough. For complex responsive designs, we use **logical operators** to combine or refine multiple conditions. Logical operators make media queries more **powerful and flexible**, allowing developers to control **when** and **how** styles apply.

Types of Logical Operators

1. **and**

- Combines multiple conditions.
- All conditions must be **true** for the styles to apply.
- Example: @media (min-width: 600px) and (orientation: landscape)

2. **not**

- Negates a media query.
- Styles apply only if the query condition is false.
- Example: @media not screen → applies to all media types **except screen**.

3. **only**

- Prevents older browsers (that don't understand media queries) from applying styles.
- Commonly used in **mobile-first design**.
- Example: @media only screen and (max-width: 768px)

4. **comma (,)** (acts as OR)

- Allows multiple queries separated by commas.
- If **any one condition is true**, the styles apply.
- Example: @media (max-width: 500px), (orientation: portrait)

Examples:

```
/* Base style */
body {
font-size: 16px;
background-color: white;
}
/* Use AND operator: screen width ≥ 768px AND orientation is portrait */
@media (min-width: 768px) and (orientation: portrait) {
body {
font-size: 20px;
background-color: lightyellow;
}
}
/* Use OR operator: width ≤ 500px OR landscape orientation */
@media (max-width: 500px), (orientation: landscape) {
body {
font-size: 14px;
background-color: lightgreen;
}
}
```

2. Describe different media types (screen, print, speech) and their implementations. Media Types in CSS

Media types specify **the general category of device** that a CSS rule applies to. They allow developers to define styles differently for screens, printers, and even assistive technologies like screen readers. The three commonly used media types are:

1. screen

- Targets devices with a **visual display** like desktops, laptops, tablets, and smartphones.
- Most common media type in responsive design.

Use case: Applying styles specifically for digital screens.

2. print

- Targets **printed documents** and print previews.
- Often removes background colors or resizes text for readability.

Use case: Creating printer-friendly styles.

3. speech

- Targets devices that **read content aloud**, such as screen readers.
- Focus is on **text content and accessibility**.

Use case: Improving experience for visually impaired users.

Program example

```
@media screen {  
body {  
background-color: lightyellow;  
}  
}  
@media print {  
body {  
background: none;  
color: black;  
font-size: 12px;  
}  
}  
@media speech {  
p {  
voice-family: male;  
stress: 20;  
richness: 90;  
}}  
}
```

3. Define minimum and maximum constraints (min-width, max-width, min-height, max height) and explain their usage with examples.

Minimum and maximum constraints in CSS control how small or large an element can be resized or displayed. They help maintain responsive and visually balanced layouts.

Min-width and min-height: These set the smallest width or height an element can shrink to, preventing it from becoming too small. **Example:**

```
div {  
min-width: 200px;  
min-height: 100px;  
}
```

The element will never be smaller than 200px wide or 100px tall.

Max-width and max-height: These set the largest width or height an element can grow to, ensuring it doesn't expand excessively.

Example:

```
img {  
  max-width: 500px;  
  max-height: 300px;  
}
```

The image will scale but won't exceed 500px width or 300px height.

Usage: These constraints are widely used in responsive web design to ensure elements adapt smoothly across screen sizes without distorting or overflowing.

4. Explain the fundamentals of CSS Media Queries and describe how they are used to create responsive web designs.

width, min-width, max-width: Relate to the width of the viewport.

height, min-height, max-height: Relate to the height of the viewport.

orientation: Checks if the device is in portrait or landscape mode.

Fundamentals of CSS Media Queries:

Syntax: Media queries begin with the @media at-rule, followed by a media type and optional media features (conditions) enclosed in parentheses. The CSS rules to be applied under these conditions are then placed within curly braces.

Example:

```
@media screen and (max-width: 768px) { /* CSS rules for screens up to 768px wide */  
body { font-size: 16px; }  
.container { width: 90%; } }
```

Media Types:

These specify the type of media the styles are intended for. Common types include:

all: Applies to all media types.

screen: Applies to computer screens, tablets, and smartphones.

print: Applies to documents viewed in print preview or printed.

speech: Applies to speech synthesizers.

Media Features (Conditions):

These are expressions that test specific characteristics of the device or viewport. Common media features include:

resolution: Checks the screen resolution.

aspect-ratio: Checks the aspect ratio of the viewport.

and: Combines multiple media features, requiring all conditions to be true.

or (represented by a comma ,): Applies styles if any of the listed conditions are true.

not: Negates a media query.

Logical Operators:

Media features can be combined using logical operators:

Role in Responsive Web Design:

Media queries are the cornerstone of Responsive Web Design because they allow developers to:

- Create Flexible Layouts:

Adjust the number of columns, element sizing, and spacing to optimize content display for different screen sizes, from mobile phones to large desktop monitors.

- Optimize Content Presentation:

Modify font sizes, image sizes, and navigation menus to ensure readability and usability across various devices.

- Enhance User Experience:

Provide a tailored experience by adapting the design based on device capabilities (e.g., larger buttons for touchscreens, different styles for print).

- **Implement Mobile-First Design:**

A common strategy involves designing for the smallest screen first and then using min-width media queries to progressively enhance the layout for larger screens. This ensures a solid base for mobile users and efficient styling for all devices.

- **Set Breakpoints:**

Define specific screen widths (breakpoints) where the layout or styling needs to change to provide an optimal viewing experience.

5. Explain the CSS Grid system in detail. Discuss grid container and grid item properties with examples.

The Grid Layout Module offers a grid-based layout system, with rows and columns.

The Grid Layout Module allows developers to easily create complex web layouts.

The Grid Layout Module makes it easy to design a responsive layout structure, without using float or positioning.

CSS Grid Components

A grid always consists of:

- **A Grid Container** - The parent (container) element, where the display property is set to grid or inline-grid
- **One or more Grid Items** - The direct children of the grid container automatically becomes grid items

All CSS Grid Properties

Property	Description
<u>align-content</u>	Vertically aligns the whole grid inside the container (when total grid size is smaller than container)
<u>align-items</u>	Specifies the default alignment for items inside a flexbox or grid container
<u>align-self</u>	Aligns the content for a specific grid item along the column axis
<u>display</u>	Specifies the display behavior (the type of rendering box) of an element
<u>column-gap</u>	Specifies the gap between the columns

CSS Grid Container

A grid container contains one or more grid items arranged in columns and rows.

All direct child elements of a grid container automatically become grid items.

An element becomes a grid container when its display property is set to grid or inline-grid.

Example

Use display: grid to make a block-level grid container:

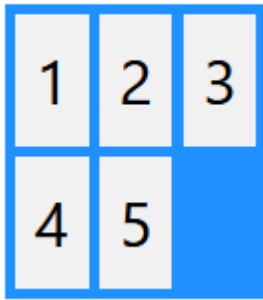
```
.container {  
  display: grid;  
}
```

Example

Use display: inline-grid to make an inline grid container:

```
.container {  
  display: inline-grid;  
}
```

Result:



CSS align-content Property

The align-content property specifies how flex lines are distributed along the cross axis in a flexbox container.

In flexbox layout, the main axis is in the [flex-direction](#) (default is 'row', horizontal), and the cross axis is perpendicular to the main axis (default is 'column', vertical).

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body {
```

```
  background-color:#E7E9EB;
```

```
}
```

```
#main {
```

```
  background-color:#FFFFFFF;
```

```
  width: 100%;
```

```
  height: 300px;
```

```
  display: flex;
```

```
  flex-wrap: wrap;
```

```
  align-content: stretch;
```

```
}
```

```
#main div {
```

```
  width: 100%;
```

```
  height: 40px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The align-content Property</h1>
```

```
<div id="main">
```

```
  <div style="background-color:coral;"></div>
```

```
  <div style="background-color:lightblue;"></div>
```

```
  <div style="background-color:pink;"></div>
```

```
</div>
```

```
</body>
```

```
</html>
```

OUTPUT:

The align-content Property



6. Write short notes on Transition Timing Functions. Explain with syntax.

Transition Timing Functions in CSS dictate the speed curve of a transition effect, determining how the animation progresses over its duration. They control the acceleration and deceleration of the animated property.

Predefined Keywords:

- **ease (default):** Starts slowly, accelerates quickly, and then decelerates at the end.
- **linear:** Maintains a constant speed throughout the transition.
- **ease-in:** Starts slowly and then accelerates.
- **ease-out:** Starts quickly and then decelerates towards the end.
- **ease-in-out:** Starts and ends slowly, with a faster pace in the middle.
- **step-start:** Jumps instantly to the final state at the beginning of the transition.
- **step-end:** Stays at the initial state until the very end, then jumps instantly to the final state.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 50px;
  background: red;
  color: white;
  font-weight: bold;
  transition: width 2s;
}

#div1 {transition-timing-function: linear;}
#div2 {transition-timing-function: ease;}
#div3 {transition-timing-function: ease-in;}
#div4 {transition-timing-function: ease-out;}
#div5 {transition-timing-function: ease-in-out;}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>The transition-timing-function Property</h1>

<div id="div1">linear</div>
<div id="div2">ease</div>
<div id="div3">ease-in</div>
```

```
<div id="div4">ease-out</div>
<div id="div5">ease-in-out</div>
```

<p>Hover over the div elements above, to see the different transition effects.</p>

```
</body>
</html>
```

OUTPUT:

The transition-timing-function Property



Hover over the div elements above, to see the different transition effects.

7. Explain briefly about animation timing functions.

Animation timing functions, also known as easing functions or speed curves, control the pace at which an animation progresses over its duration.

They define how the animation's speed changes from its start to its end, influencing the "feel" of the movement.

Instead of a constant, linear speed, timing functions allow for more dynamic and natural-looking animations by introducing acceleration and deceleration.

This is achieved by mapping the animation's progress over time using various mathematical curves.

Value	Description
linear	The animation has the same speed from start to end
ease	Default value. The animation has a slow start, then fast, before it ends slowly
ease-in	The animation has a slow start
ease-out	The animation has a slow end
ease-in-out	The animation has both a slow start and a slow end
step-start	Equivalent to steps(1, start)
step-end	Equivalent to steps(1, end)
<u>steps(int,step-position)</u>	Specifies a stepping function, with two parameters. The first parameter specifies the number of steps/intervals. The second parameter, specifies when the jump between values occurs
<u>cubic-bezier(n,n,n,n)</u>	Define your own values in the cubic-bezier function Possible values are numeric values from 0 to 1
initial	Sets this property to its default value. Read about initial
inherit	Inherits this property from its parent element. Read about inherit

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
```

```
div {
  width: 100px;
  height: 50px;
  background: red;
  color: white;
  font-weight: bold;
  position: relative;
  animation: mymove 5s infinite;
}

#div1 {animation-timing-function: linear;}
#div2 {animation-timing-function: ease;}
#div3 {animation-timing-function: ease-in;}
#div4 {animation-timing-function: ease-out;}
#div5 {animation-timing-function: ease-in-out;}

@keyframes mymove {
  from {left: 0px;}
  to {left: 300px;}
}
</style>
</head>
<body>
```

<h1>Elements with Different timing function Values</h1>

```
<div id="div1">linear</div>
<div id="div2">ease</div>
<div id="div3">ease-in</div>
<div id="div4">ease-out</div>
<div id="div5">ease-in-out</div>

</body>
</html>
```

OUTPUT:

Elements with Different timing function Values



UNIT-4

SHORT QUESTIONS:

1. What is parameter passing?

Parameter passing is the process of providing inputs (arguments) to a function through its parameters, allowing the function to work with those values.

Example:

```
function add(x, y) {    // x and y are parameters  
    return x + y; }
```

2. Define innerText and innerHTML.

Ans: **innerText** is a property in JavaScript used to get or set **the visible text content** of an element.

Key Points:

- It returns **only the text that is visible** (not hidden by CSS).
- Ignores HTML tags.
- Triggered by layout changes (reflow), so it is slower.

innerHTML is a property in JavaScript used to get or set **the HTML content** inside an element.

Key Points:

- Returns both **text + HTML tags**.
- Allows you to insert HTML elements.
- Faster because it does not consider CSS visibility.

3. What is a conditional statement?

Ans: Conditional statements in JavaScript enable code to make decisions and execute different blocks of code based on whether a specified condition evaluates to true or false.

Example:

if statement: Executes a block of code only if a specified condition is true.

if...else statement: Executes one block of code if the condition is true, and a different block of code if the condition is false.

4. What is attribute manipulation? Give an example.

Ans: Attribute manipulation is the process of changing or altering properties of data or an object, such as modifying an image's facial expression, changing a website's class, or updating a specific data field in a database.

Attribute Manipulation (using JavaScript):

```
let myLink = document.getElementById("myLink");
```

```
myLink.setAttribute("href", "new-page.html"); Resulting
```

HTML:

html

```
<a id="myLink" href="new-page.html">Click me</a>
```

5. What are logical operators? Give examples

Ans: Logical operators are used to **combine boolean expressions**. Logical operators can be used to modify the results of comparisons. Typically, you will use a comparison operator to check a condition, and a logical operator to combine conditions into more complex logic. Logical operators are used to determine the logic between variables or values.

Given that **x = 6 and y = 3**, the table below explains the logical operators:

Oper	Name	Example
&&	AND	(x < 10 && y > 1) is true
	OR	(x === 5 y === 5) is false
!	NOT	!(x === y) is true

6. What is DOM?

7. Write any two element selection methods in DOM.

Ans: document.getElementById(): This method selects a single element based on its unique id attribute. Since id attributes are meant to be unique within an HTML document, this method is efficient for directly targeting a specific element.

document.getElementsByClassName(): This method selects all elements that share a specified class name. It returns a live HTMLCollection (an array-like object) of all matching elements in the order they appear in the document

8. What is textContent?

The textContent property is a standard JavaScript DOM (Document Object Model) interface used to get or set the raw textual content of a node and all its descendants. It provides access to the text as it appears in the HTML source code, including spacing, line breaks, and text from elements that might be hidden by CSS (e.g., display: none;).

9. What are let, var, and const?

var, let, and const are JavaScript keywords for declaring variables, differing mainly in scope, reassignment, and redeclaration rules; var has function scope and allows reassignment/redeclaration, let offers block scope and reassignable values, while const provides block scope and requires a constant, unchanging value (no reassignment/redeclaration) after initialization, making let and const (introduced in ES6) modern best practices for clearer, more predictable code.

```
function exampleVar() {  
  var x = 10;  
}
```

LONG QUESTIONS:

1. Describe JavaScript data types with examples.

Ans : In programming, data types is an important concept. To be able to operate on variables, it is important to know something about the type. Without data types, a computer cannot safely solve this:

```
let x = 16 + "Volvo";
```

JavaScript utilizes various data types to represent different kinds of values. These can be broadly categorized into primitive and non-primitive (or reference) types.

Type	Description
String	A text of characters enclosed in quotes
Number	A number representing a mathematical value
Bigint	A number representing a large integer
Boolean	A data type representing true or false
Object	A collection of key-value pairs of data
Undefined	A primitive variable with no assigned value
Null	A primitive value representing object absence
Symbol	A unique and primitive identifier

Examples

// Strings

```
let color = "Yellow";
```

```
let lastName = "Johnson";
```

// Number

```
let length = 16; let
```

```
weight = 7.5;
```

// BigInt

```
let x = 1234567890123456789012345n;
```

```
let y = BigInt(1234567890123456789012345)
```

// Boolean let x

```
= true; let y =
```

```
false;
```

// Object

```
const person = {firstName:"John", lastName:"Doe"};
```

// Array object

```
const cars = ["Saab", "Volvo", "BMW"];
```

// Date object

```
const date = new Date("2022-03-25");
```

// Undefined let

```
x;
```

```
let y;
```

```
// Null
```

```
let x = null; let
```

```
y = null;
```

```
// Symbol
```

```
const x = Symbol(); const
```

```
y = Symbol();
```

JavaScript Types are Dynamic

JavaScript has dynamic types. This means that the same variable can be used to hold different data types:

Example

```
let x;    // Now x is undefined x = 5;
```

```
// Now x is a Number x = "John"; //
```

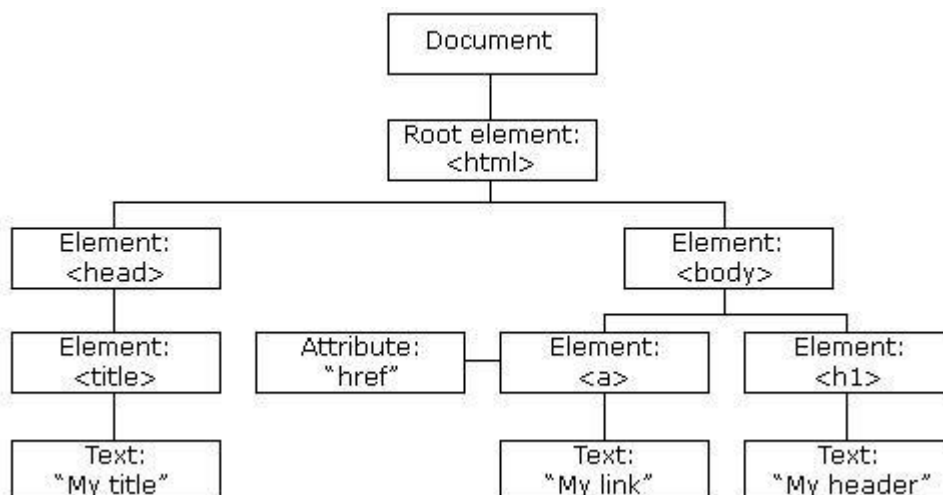
```
Now x is a String
```

2. Explain DOM hierarchy in detail. Describe parent, child, and sibling relationships with examples.

Ans : DOM Nodes

According to the W3C HTML DOM standard, everything in an HTML document is a node:

- The entire document is a document node
- Every HTML element is an element node
- The text inside HTML elements are text nodes
- Every HTML attribute is an attribute node (deprecated)
- All comments are comment nodes



Node Relationships

The nodes in the node tree have a hierarchical relationship to each other. The terms

parent, child, and sibling are used to describe the relationships.

- In a node tree, the top node is called the root (or root node)
- Every node has exactly one parent, except the root (which has no parent)
- A node can have a number of children
- Siblings (brothers or sisters) are nodes with the same parent
- `<html>`

```
<head>
  <title>DOM Tutorial</title>
</head>
```

```
<body>
  <h1>DOM Lesson one</h1>
  <p>Hello world!</p>
</body>
```

```
</html>
```

From the HTML above you can read:

- `<html>` is the root node
- `<html>` has no parents
- `<html>` is the parent of `<head>` and `<body>`
- `<head>` is the first child of `<html>`
- `<body>` is the last child of `<html>`

and:

- `<head>` has one child: `<title>`
- `<title>` has one child (a text node): "DOM Tutorial"
- `<body>` has two children: `<h1>` and `<p>`
- `<h1>` has one child: "DOM Lesson one"
- `<p>` has one child: "Hello world!"
- `<h1>` and `<p>` are siblings

3. Explain different types of loops in JavaScript with examples.

Ans : JavaScript loops **repeat executing a block of code** a number of times.

Loops are fundamental for tasks involving:

- Iteration over values
- Iteration over data structures
- Performing an action multiple times

Loop Type	Description
for	Iterates over values and expressions
while	Iterates over a condition
do...while	Iterates over a condition
for...in	Iterates over the properties of an Object
for...of	Iterates over array like objects
forEach()	Iterates over each element in an Array

Example; for loop

```

<!DOCTYPE html>
<html>
<body>
< h1>JavaScript Loops</h1>
<h2>The for Loop</h2>

<p id="demo"></p>

<script>
let text = "";

for (let i = 0; i < 5; i++) {
    text += "The number is " + i + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

OUTPUT:

JavaScript Loops

The for Loop

```
The number is 0  
The number is 1  
The number is 2  
The number is 3  
The number is 4
```

Example: while loop

```
<!DOCTYPE html>  
<html>  
<body>  
<h1>JavaScript Loops</h1>  
<h2>The While Loop</h2>  
<p id="demo"></p>  
<script>  
let text = "";  
  
let i = 0;  
while (i < 10) {  
    text += "The number is " + i + "<br>"; i++;  
}  
  
document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

OUTPUT:

JavaScript Loops

The While Loop

```
The number is 0  
The number is 1  
The number is 2  
The number is 3  
The number is 4  
The number is 5  
The number is 6  
The number is 7  
The number is 8  
The number is 9
```

The for...in Loop

The for...in loop iterates over the enumerable properties of an object. It is typically used for iterating over object keys.

Example:

```
<!DOCTYPE html>
<html>
<body>
< h1>JavaScript Loops</h1>
<h2>The for in Loop</h2>

<p>A for in statement loops through the properties of a person object:</p>

<p id="demo"></p>

<script>
const person = { fname:"John", lname:"Doe", age:25};

let txt = "";
  for (let x in person) { txt
    += person[x] + " ";
  }

document.getElementById("demo").innerHTML = txt;
</script>

</body>
</html>
```

OUTPUT:

JavaScript Loops

The for in Loop

A for in statement loops through the properties of a person object:

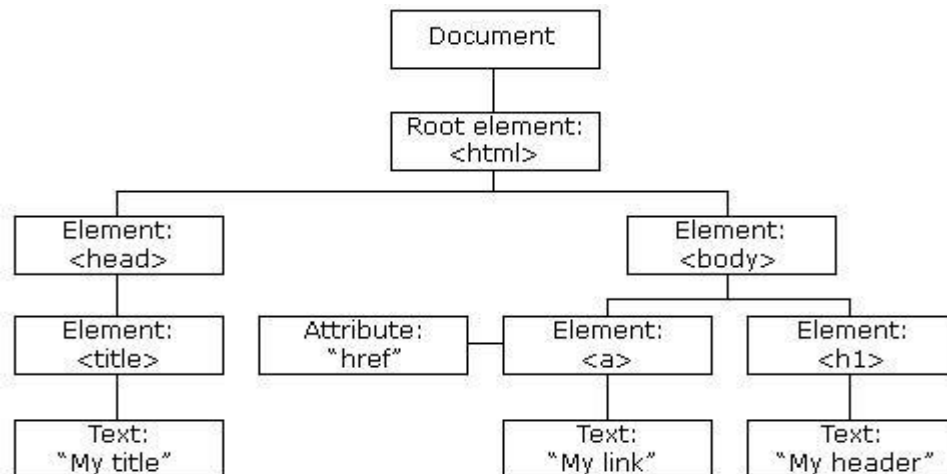
John Doe 25

4. Explain the Document Object Model (DOM) in detail. Draw and explain the DOM tree structure.

Ans : When a web page is loaded, the browser creates a **Document Object Model** of the page. With the

HTML DOM, JavaScript can access and change all the elements of an HTML document.

The **HTML DOM** model is constructed as a tree of **Objects**:



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

Example:

```
<html>
```

```
<head>
```

```
<title>DOM Tutorial</title>
```

```
</head>
```

```
<body>
```

```
<h1>DOM Lesson one</h1>
```

```
<p>Hello world!</p>
```

```
</body>
```

</html>

From the HTML above you can read:

- <html> is the root node
- <html> has no parents
- <html> is the parent of <head> and <body>
- <head> is the first child of <html>
- <body> is the last child of <html>

and:

- <head> has one child: <title>
- <title> has one child (a text node): "DOM Tutorial"
- <body> has two children: <h1> and <p>
- <h1> has one child: "DOM Lesson one"
- <p> has one child: "Hello world!"

<h1> and <p> are siblings

5. Explain different types of control structures in JavaScript.

Ans : Control Flow is the order in which statements are executed in a program.

By default, JavaScript runs code **from top to bottom** and left to right.

Control flow statements let you **change that order**, based on conditions, loops or keywords.

Conditional Control Flow

Conditions let you **make decisions** using:

- a. if
- b. if...else
- c. switch
- d. ternary (? :)

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
    <h1>JavaScript if Statement</h1>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let age = 18;
```

```
let text = "Unknown"
```

```
if (age >= 18) { text  
    = "Adult";
```

```
} else {  
    text = "Minor";  
}
```

```
document.getElementById("demo").innerHTML = "The group is " + text;  
</script>  
</body>  
</html>
```

Loops (Repetition Control Flow)

Loops let you **run code multiple times** using:

- e. for
- f. while
- g. do...while

Example:

```
<!DOCTYPE html>  
<html>  
<body>  
<h1>JavaScript Loops</h1>  
<h2>The for Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>  
let text = "";  
for(let i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}
```

```
document.getElementById("demo").innerHTML = text;  
</script>
```

```
</body>  
</html>
```

Jump Statements

Jump statements let you **change the flow** abruptly using:

- h. break - exits a loop or switch
- i. continue - skips the current loop iteration
- j. return - exits from a function

k. throw - jumps to error handling

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript break</h1>
```

```
<h2>The break Statement in a Loop</h2>
```

```
<p>Break the loop when the loop counter is 3:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "";
```

```
for (let i = 0; i < 10; i++) { if (i  
    === 3) { break; }  
    text += "The number is " + i + "<br>";  
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

UNIT-5

1. Define event propagation

Ans: **Event propagation** is the mechanism that controls how events move through the DOM, either from the top to the target element (**capturing phase**) or from the target element back up (**bubbling phase**).

- Capturing Phase (Event Capture)
- Target Phase
- Bubbling Phase

2. Write syntax of Fetch API for a GET request.

Ans: `fetch('https://example.com/api/data')`
`.then(response => response.json())`
`.then(data => { console.log(data);`
`})`
`.catch(error => {`
 `console.error('Error:', error);`
`});`

3. What is event listener?

An **event listener** in JavaScript is a function or method that waits for a specific action or occurrence (an "event") to happen on a web page element and then executes a specified code in response. This mechanism allows developers to make web pages interactive and dynamic, responding to user actions like clicks, key presses, or mouse movements.

Key Concepts

- **Events:** Common events include click, mouseover, keydown, submit, and load.
- **Event Handler (Callback Function):** This is the function containing the code that runs when the event the listener is "watching" for occurs.
- **Target:** The HTML element (or other DOM object like document or window) to which the event listener is attached.

4. What is form validation?

Ans: Form validation is the process of checking user input in a web form to ensure it meets specific criteria and requirements before it is submitted to a server or processed further. It is a crucial part of web development that aims to improve data quality, enhance user experience, and bolster security.

Key aspects of form validation include:

Ensuring completeness, Validating data format, Enforcing constraints, Maintaining consistency and Preventing malicious input.

5. What are JSON objects and JSON arrays?

Ans: A JSON object represents an unordered collection of key-value pairs. It is enclosed in curly braces {}

Values: Can be any valid JSON data type: string, number, boolean (true or false), null, another JSON object, or a JSON array.

```
{
  "name": "Alice",
  "age": 30,
  "isStudent": false,
  "address":
  {
    "street": "123 Main St",
    "city": "Anytown"
  }
}
```

JSON Array:

A JSON array represents an ordered collection of values. It is enclosed in square brackets []

Values: Can be any valid JSON data type: string, number, boolean, null, a JSON object, or another JSON array.

```
[
  "apple",
  "banana", "orange"
]
```

6. List any four input element types.

Ans: Here are four common input element types used in HTML forms:

- **Text:** Used for single-line text input.
- **Password:** Used for password input, masking the characters as they are typed.
- **Email:** Used for entering email addresses, often including validation for a valid email format.
- **Number:** Used for entering numerical values, often with built-in controls for incrementing/decrementing.

7. What is the difference between XML and JSON?

JSON	XML
It is JavaScript Object Notation	It is Extensible markup language

It is based on JavaScript language.	It is derived from SGML.
It is a way of representing objects.	It is a markup language and uses tag structure to represent data items.
It does not provides any support for namespaces.	It supports namespaces.
It supports array.	It doesn't supports array.
Its files are very easy to read as compared to XML.	Its documents are comparatively difficult to read and interpret.
It doesn't use end tag.	It has start and end tags.
It is less secured.	It is more secured than JSON.

8. Define callbacks.

A callback in JavaScript is a function passed as an argument to another function, which is then executed (or "called back") at a later point in time. This mechanism is fundamental to JavaScript's event-driven and asynchronous programming model.

Key Concepts

- **Function as an Argument:** In JavaScript, functions are "first-class citizens," meaning they can be treated like any other variable or object, including being passed as arguments to other functions.
- **Deferred Execution:** The core idea is to define a behavior now, but execute it later, when a specific event occurs or a task completes.
- **Enabling Asynchronous Behavior:** Callbacks are crucial for handling non-blocking operations like fetching data from a server, setting timers, or listening for user events (like button clicks), allowing the rest of the code to continue running without waiting.

9. What are asynchronous functions?

Asynchronous functions are special functions that run non-blocking operations, allowing a program to remain responsive while waiting for long tasks (like network requests or file I/O) to complete, instead of halting everything. They use keywords like `async` and `await` (in languages like JavaScript/Dart) to make asynchronous code look and behave more like synchronous code, returning a `Promise` or `Future` that resolves with the result later, enabling efficient handling of I/O-bound operations without freezing the application.

Key Characteristics

6. **Non-Blocking:** They initiate a task and then move on to other work, rather than waiting for the task to finish.
7. **Return a Promise/Future:** An `async` function always returns a promise (or `Future` in Dart) that will eventually resolve with the function's value or reject with an error.
8. **`await` Keyword:** Inside an `async` function, `await` pauses execution within that function until the promise it's waiting on settles (fulfills or rejects), simplifying complex asynchronous logic.
9. **Syntactic Sugar:** `async/await` is built on top of promises, providing a cleaner, more readable way to write asynchronous code.

LONG QUESTIONS

1.Explain node properties in JavaScript DOM. Provide suitable examples.

Ans : In JavaScript's Document Object Model (DOM), everything within an HTML document is represented as a node. These nodes possess various properties that allow for interaction and manipulation of the document's structure, content and style.

Common Node Properties:

nodeType: Returns a number representing the type of the node.

- 1 for Element nodes (e.g., <div>, <p>).
- 2 for Attribute nodes (e.g., href in).
- 3 for Text nodes (e.g., the text content within a <p> tag).
- 9 for Document nodes (the entire HTML document).

nodeName: Returns the name of the node.

- For Element nodes, it's the tag name in uppercase (e.g., "DIV", "P").
- For Text nodes, it's "#text".
- For Document nodes, it's "#document".

nodeValue: Returns or sets the value of the node.

- For Text nodes, it's the actual text content.
- For Attribute nodes, it's the attribute's value.
- For Element nodes, it's null.

parentNode: Returns the parent node of the current node. If there is no parent (e.g., for the Document node), it returns null.

childNodes: Returns a live NodeList of all child nodes of the current node, including text nodes and comment nodes.

firstChild: Returns the first child node of the current node.

lastChild: Returns the last child node of the current node.

nextSibling: Returns the next sibling node in the same parent's childNodes list.

previousSibling: Returns the previous sibling node in the same parent's childNodes list.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript HTML DOM</h2>
<p>Add a new HTML Element.</p>

<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>
```

```
<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new."); para.appendChild(node);
const element = document.getElementById("div1"); element.appendChild(para);
</script>

</body>
</html>
```

OUTPUT:

JavaScript HTML DOM

Add a new HTML Element.

This is a paragraph.

This is another paragraph.

This is new.

2.Explain the structure of an HTML form. Describe input element types with examples

Ans : An HTML form is used to collect user input. The user input is most often sent to a server for processing. The HTML `<form>` element is used to create an HTML form for user input:

The `<form>` element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

The `<input>` Element

The HTML `<input>` element is the most used form element.

An `<input>` element can be displayed in many ways, depending on the type attribute. Here are some examples:

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="submit"></code>	Displays a submit button (for submitting the form)
<code><input type="button"></code>	Displays a clickable button

HTML Input Types

Here are the different input types you can use in HTML:

- `<input type="button">`
- `<input type="checkbox">`
- `<input type="color">`
- `<input type="date">`
- `<input type="datetime-local">`
- `<input type="email">`
- `<input type="file">`
- `<input type="hidden">`
- `<input type="image">`
- `<input type="month">`
- `<input type="number">`
- `<input type="password">`

Input Type Text

`<input type="text">` defines a single-line text input field: Example

`<form>`

`<label for="fname">First name:</label><br>`

`<input type="text" id="fname" name="fname"><br>`

`<label for="lname">Last name:</label><br>`

`<input type="text" id="lname" name="lname">`

`</form>`

OUTPUT:

This is how the HTML code above will be displayed in a browser:

First name:

Last name:

Input Type Password

`<input type="password">` defines a **password field**:

Example

`<form>`

`<label for="username">Username:</label><br>`

`<input type="text" id="username" name="username"><br>`

`<label for="pwd">Password:</label><br>`

`<input type="password" id="pwd" name="pwd">`

</form>

OUTPUT:

This is how the HTML code above will be displayed in a browser:

Username:

Password:

Input Type Submit

`<input type="submit">` defines a button for **submitting** form data to a **form-handler**. The form-handler is typically a server page with a script for processing input data.

The form-handler is specified in the form's action attribute:

Example

```
<form action="/action_page.php">  
  <label for="fname">First name:</label><br>  
  <input type="text" id="fname" name="fname" value="John"><br>  
  <label for="lname">Last name:</label><br>  
  <input type="text" id="lname" name="lname" value="Doe"><br><br>  
  <input type="submit" value="Submit">  
</form>
```

OUTPUT:

This is how the HTML code above will be displayed in a browser:

First name:

Last name:

3.Explain JSON parsing and serialization in detail. Discuss use cases, examples, and the importance in client–server communication.

Ans :JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is human-readable and easy for machines to parse and generate. JSON is built on two structures: a collection of name/value pairs (object) and an ordered list of values (array).

JSON Parsing (Deserialization):

JSON parsing is the process of converting a JSON-formatted string into a native data structure or object in a specific programming language. When data is received from an external source, such as a web server, it typically arrives as a JSON string. Parsing allows the application to access and manipulate this data using the language's native data types.

1. JSON.stringify()

Imagine we have this object in JavaScript:

```
const obj = {name: "John", age: 30, city: "New York"};
```

Use the JavaScript function `JSON.stringify()` to convert it into a string. `const myJSON = JSON.stringify(obj);`

The result will be a string following the JSON notation. `myJSON` is now a string, and ready to be sent to a server:

Example

```
const obj = {name: "John", age: 30, city: "New York"}; const  
myJSON = JSON.stringify(obj);
```

2. JSON.parse()

Example - Parsing JSON

Imagine we received this text from a web server: `'{"name":"John", "age":30,`

```
"city":"New York"}'
```

Use the JavaScript function `JSON.parse()` to convert text into a JavaScript object: `const obj =`

```
JSON.parse('{"name":"John", "age":30, "city":"New York"}');
```

Example

```
'{"name":"John", "age":30, "car":null}'
```

The example above defines an object with 3 properties:

- "name"
- "age"
- "car"

Each property has a value:

- "John"
- 30
- null

Use Cases:

- **Web APIs:** JSON is the predominant format for data exchange in web services and RESTful APIs.
- **Configuration Files:** Many applications use JSON for storing configuration settings due to its human-readable nature.
- **Data Interchange:** Facilitating data transfer between different systems and programming languages.
- **Client-Server Communication:** Enabling web clients (browsers, mobile apps) to interact with server-side applications.
- **NoSQL Databases:** Databases like MongoDB store data in JSON-like documents, making JSON serialization and parsing crucial.

JSON's role in client-server communication is paramount:

- **Standardization:** Provides a universal, language-agnostic format for data exchange, ensuring interoperability between diverse systems.
- **Readability and Simplicity:** Its human-readable syntax simplifies debugging and development.
- **Efficiency:** While not as compact as binary formats, JSON is generally efficient enough for most web-based data transfer and is easily processed by client-side JavaScript.
- **Flexibility:** Supports complex nested data structures, allowing for rich and detailed data representation.
- **Wide Adoption:** Extensive library support across virtually all programming languages simplifies implementation.

4.Explain scheduler functions: setTimeout() and setInterval() with examples

Ans : The window object allows execution of code at specified time intervals.

These time intervals are called timing events.

The two key methods to use with JavaScript are:

- `setTimeout(function, milliseconds)`
Executes a function, after waiting a specified number of milliseconds.
- `setInterval(function, milliseconds)`
Same as `setTimeout()`, but repeats the execution of the function continuously.

The `setTimeout()` and `setInterval()` are both methods of the HTML DOM Window object.

The setTimeout() Method

```
window.setTimeout(function, milliseconds);
```

The `window.setTimeout()` method can be written without the window prefix. The first parameter is a function to be executed.

The second parameter indicates the number of milliseconds before execution.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Timing</h2>

<p>Click "Try it". Wait 3 seconds, and the page will alert "Hello".</p>

<button onclick="setTimeout(myFunction, 3000);">Try it</button>

<script>
  function myFunction() { alert('Hello');
  }
</script>

</body>
</html>
```

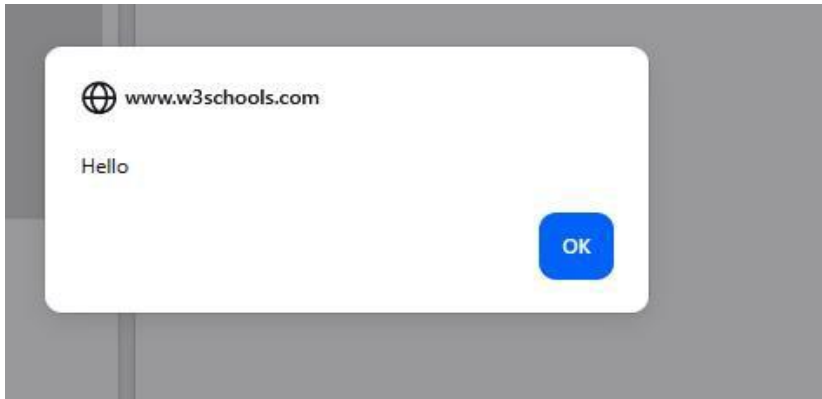
OUTPUT:

JavaScript Timing

Click "Try it". Wait 3 seconds, and the page will alert "Hello".

Try it

After “click”



The setInterval() Method

The setInterval() method repeats a given function at every given time-interval.

window.setInterval(function, milliseconds);

The window.setInterval() method can be written without the window prefix. The first parameter is the function to be executed.

The second parameter indicates the length of the time-interval between each execution. This example executes a function called "myTimer" once every second (like a digital watch).

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Timing</h2>
```

```
<p>A script on this page starts this clock:</p>
```

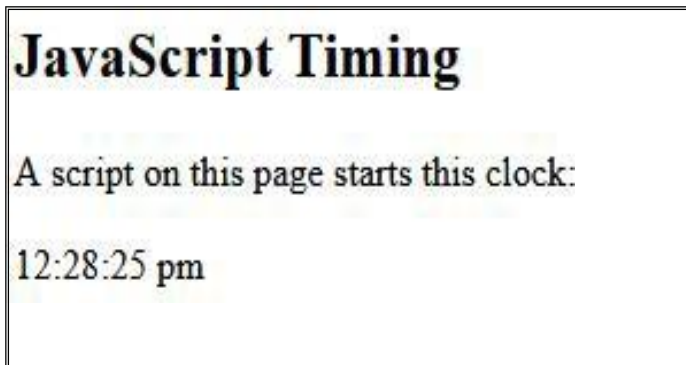
```
<p id="demo"></p>
```

```
<script> setInterval(myTimer,
1000);

function myTimer() { const d
= new Date();
document.getElementById("demo").innerHTML = d.toLocaleTimeString();
}
</script>

</body>
</html>
```

OUTPUT:



5.Explain HTTP communication. Describe request methods, status codes, and headers.

Ans : HTTP communication operates as a request-response protocol between a client (e.g., a web browser) and a server. The client sends an HTTP request to the server, and the server responds with an HTTP response.

Request Methods

HTTP request methods, also known as verbs, indicate the desired action to be performed on a resource:

- **GET:** Retrieves data from a specified resource.
- **POST:** Submits data to be processed to a specified resource, often creating a new resource.

- **PUT:** Replaces all current representations of the target resource with the request payload. If the resource does not exist, it may create it.
- **PATCH:** Applies partial modifications to a resource.
- **DELETE:** Deletes the specified resource.
- **HEAD:** Similar to GET, but requests only the headers of a resource, not the body.
- **OPTIONS:** Describes the communication options for the target resource.

Status Codes

HTTP status codes are three-digit numbers returned by the server in the response, indicating the outcome of the request. They are grouped into five classes:

- **1xx Informational:** Request received, continuing process. (e.g., 100 Continue)
- **2xx Success:** The action was successfully received, understood, and accepted. (e.g., 200 OK, 201 Created)
- **3xx Redirection:** Further action needs to be taken to complete the request. (e.g., 301 Moved Permanently, 302 Found)
- **4xx Client Error:** The request contains bad syntax or cannot be fulfilled. (e.g., 400 Bad Request, 404 Not Found)
- **5xx Server Error:** The server failed to fulfill an apparently valid request. (e.g., 500 Internal Server Error, 503 Service Unavailable)

Headers

HTTP headers are key-value pairs that carry metadata about the request or response. They provide information about the client, server, or the data being exchanged.

Common Request Headers:

- **User-Agent:** Identifies the client software.
- **Accept:** Specifies media types the client can handle.
- **Content-Type:** Indicates the media type of the request body.
- **Authorization:** Carries authentication credentials.

Common Response Headers:

- **Content-Type:** Indicates the media type of the response body.
- **Content-Length:** The size of the response body in bytes.
- **Cache-Control:** Directives for caching mechanisms.

- Set-Cookie: Sends cookies from the server to the client.

6. Describe the JavaScript event model with a neat diagram.

Ans : The JavaScript event model describes how events (user actions or browser actions) are handled and propagated within the Document Object Model (DOM). It involves three main phases: Capturing, Target, and Bubbling.

1. Capturing Phase:

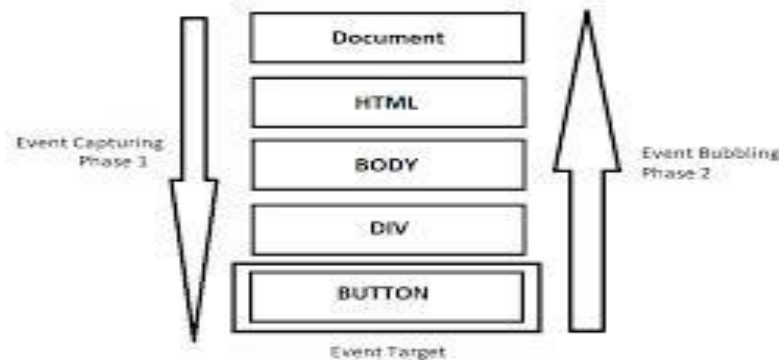
The event starts at the window object and travels down the DOM tree to the target element. During this phase, event listeners registered for the capturing phase on parent elements will be triggered.

2. Target Phase:

The event reaches the target element where the event originally occurred. Event listeners registered directly on this element for either capturing or bubbling will be executed.

3. Bubbling Phase:

The event then travels back up the DOM tree from the target element to the window object. During this phase, event listeners registered for the bubbling phase on parent elements will be triggered.



Here is a list of some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

7.Explain the structure of an HTTP request and HTTP response with diagrams.

Ans : HTTP requests and responses are fundamental to web communication, following a distinct structure for efficient information exchange between clients and servers.

HTTP Request Structure

An HTTP request consists of three main parts:

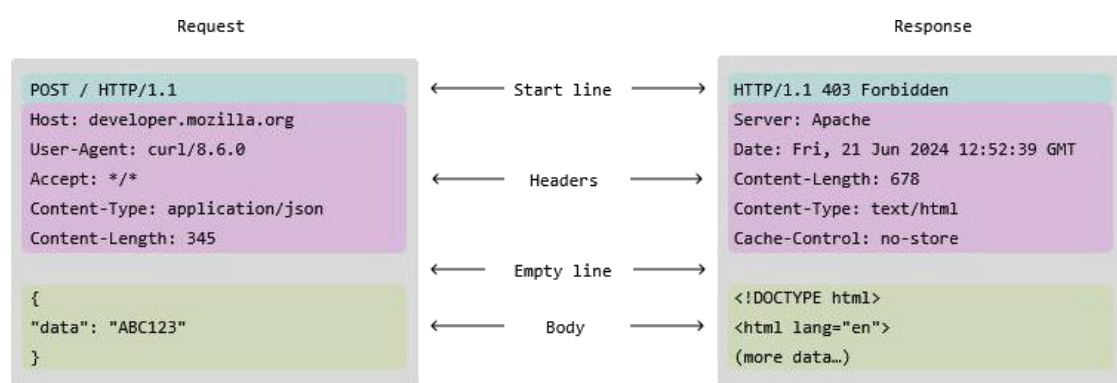
Request Line: This is the first line of the request and contains:

- **Method:** The HTTP method (e.g., GET, POST, PUT, DELETE) indicating the action to be performed.
- **Request-URI:** The Uniform Resource Identifier (URI) identifying the target resource.
- **HTTP Version:** The version of the HTTP protocol being used (e.g., HTTP/1.1).

Request Headers: These lines follow the request line and provide additional information about the request, the client, or the desired response. Examples include:

- Host: The domain name of the server.
- User-Agent: Information about the client (browser and operating system).
- Accept: Preferred media types for the response.
- Content-Type: The media type of the request body (if present).

Request Body (Optional): This section contains data being sent to the server, primarily used with methods like POST or PUT for submitting form data or uploading files. It is separated from the headers by an empty line.



HTTP Response Structure

An HTTP response also consists of three main parts:

- **Status Line:** The first line of the response, containing:
 - **HTTP Version:** The version of the HTTP protocol used by the server.
 - **Status Code:** A three-digit code indicating the outcome of the request (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
 - **Reason Phrase:** A brief, human-readable description of the status code.

Response Headers: These lines follow the status line and provide information about the response, the server, or the content being sent. Examples include:

- **Content-Type:** The media type of the response body.
- **Content-Length:** The size of the response body in bytes.
- **Server:** Information about the server software.
- **Date:** The date and time the response was generated.

Response Body (Optional): This section contains the actual data or content requested by the client (e.g., HTML, images, JSON data). It is separated from the headers by an empty line.